

Sistema de Memoria y Aprendizaje

Capa cognitiva persistente para agentes de IA

Version 1.0 | May 2026

Proyecto: brain-ai-01

Basado en: Artemis v3, GPT-5, MinMax-M2.7
Stack: Python | ChromaDB | FastAPI | Pydantic

Indice

- 1 Proposito**
- 2 Arquitectura de Memoria**
 - 2.1 Memoria Episodica
 - 2.2 Memoria Semantica
 - 2.3 Sistema de Recuperacion
- 3 Ciclo de Aprendizaje**
- 4 Scoring Hibrido**
- 5 APIs MCP (FastAPI)**
- 6 Mejoras Futuras**
- 7 Estructura de Datos**
- 8 Comandos de Uso**

1. Proposito

Este sistema es una capa cognitiva persistente que se situa encima de cualquier LLM. No es un 'chatbot con memoria', sino un motor de aprendizaje continuo que permite que un agente de IA acumule experiencia, la consolide en conocimiento reutilizable, y mejore con el tiempo independientemente del modelo subyacente.

Resuelve cinco problemas fundamentales de forma simultanea:

Problema	Solucion
Persistencia	Datos en JSON + ChromaDB vectorial
Recuperacion eficiente	Busqueda hibrida (vectores + metadatos + scoring)
Consolidacion / aprendizaje	Episodios -> Resumen -> Factcheck -> Semantico
Gobernanza / confianza	Trazabilidad, politicas, redaccion PII, evaluaciones
Compatibilidad multi-modelo	FastAPI MCP, compatible con OpenCode/Claude/Cursor

2. Arquitectura de Memoria

El sistema implementa tres niveles de memoria, inspirados en el modelo cognitivo humano. Cada nivel tiene un proposito, ciclo de vida y mecanismo de persistencia especifico.

2.1 Memoria Episodica (experiencias crudas)

La memoria episodica registra eventos, decisiones, acciones y resultados tal como ocurren. Es la materia prima del sistema.

- Ubicacion: memory/episodic/*.json
- Indexada en ChromaDB (coleccion episodic)

Almacena conocimiento destilado y validado: hechos, decisiones importantes, patrones recurrentes y errores aprendidos.

- Ubicacion: memory/semantic/*.json
- Indexada en ChromaDB (coleccion semantic)

Combina busqueda vectorial con filtros de metadatos y scoring hibrido.

- Embedding: hash determinista offline (1536d) u OpenAI API
- Filtros: proyecto, tags, rango de fechas

+-----+ SISTEMA DE MEMORIA (3 niveles) +-----+		
MEMORIA EPISODICA (experiencias crudas)		
memory/episodic/*.json		
- Indexada en ChromaDB (coleccion episodic)		
- Ciclo de vida: 24 meses		
+-----+		
Consolidacion (confidence >= 0.6)		
+-----+		
MEMORIA SEMANTICA (conocimiento consolidado)		
memory/semantic/*.json		
- facts, decisions, patterns, mistakes		
- Ciclo de vida: indefinido		
+-----+		
Retrieval hibrido (consulta -> ranking)		
+-----+		
SISTEMA DE RECUPERACION		
- Scoring: 0.45*vec + 0.2*rec + 0.2*ev + 0.15*conf		
- Filtros: proyecto, tags, rango de fechas		
+-----+		

3. Ciclo de Aprendizaje

Cada experiencia entrante pasa por un pipeline de 6 etapas que la transforma de dato crudo a conocimiento consolidado y reusable.

Etapas 1: Ingesta

Entrada: episodio JSON con campos requeridos (project, source_type, author, title, summary, timestamp). Se valida el schema, se redacta PII (emails, tokens, telefonos), se genera embedding_text, se persiste en memory/episodic/ y se indexa en ChromaDB.

Etapas 2: Resumen

Se extrae del episodio un resumen estructurado con titulo, decisiones, acciones y riesgos. Se guarda en memory/summaries/.

Etapas 3: Reflexion

Opcional. Analiza en batch los episodios de un proyecto para detectar patrones (tags mas frecuentes) y temas recurrentes. Se guarda en memory/reflections/.

Etapas 4: Factcheck

Busca en la memoria semantica decisiones similares. Detecta contradicciones (ej. usar X vs no usar X). Confidence inicial: 0.8 base, -0.3 si hay contradiccion, -0.2 si no hay evidencia.

Etapas 5: Consolidacion

Si confidence ≥ 0.6 y hay ≥ 1 evidencia, se promueve la decision a memoria semantica. Si ya existe una decision identica, se incrementa su confidence en +0.05 y se agrega la nueva fuente de evidencia.

Etapas 6: Evaluacion

Mide calidad del retrieval: relevance (0-1), has_evidence (bool), hallucination_risk (0-1). Resultados en logs/evaluations/.

```
+-----+
|  INGESTA  |
+-----+
|
Validation + Redaccion PII + Embedding
|
+-----v-----+
|  RESUMEN  |  -> memory/summaries/
+-----+
|
+-----v-----+
|  REFLEXION  |  -> memory/reflections/
+-----+
|
+-----v-----+
|  FACTCHECK  |  -> logs/traces/
+-----+
|
confidence >= 0.6?
|
+-----v-----+
|  CONSOLIDACION  |  -> memory/semantic/
+-----+  -> ChromaDB index
|
+-----v-----+
|  RETRIEVAL  |  Busqueda hibrida
+-----+
|
+-----v-----+
|  EVALUACION  |  -> logs/evaluations/
+-----+
```

4. Scoring Hibrido

Asigna un peso a cada resultado de retrieval combinando cuatro factores para determinar la relevancia real de cada ítem de memoria.

Formula

```
score = 0.45 * similitud_vectorial
        + 0.20 * recencia
        + 0.20 * cantidad_evidencia
        + 0.15 * confianza_historica
```

Factores

Factor	Peso	Descripcion	Rango
Similitud vectorial	0.45	Coseno entre embedding query y doc	0.0 - 1.0
Recencia	0.20	Antigüedad del ítem en días	0.2 - 1.0
Evidencia	0.20	Cantidad de fuentes que respaldan	0.5 - 1.0
Confianza	0.15	Score de confianza del ítem	0.0 - 1.0

Recencia (detalle)

Antigüedad	Score
< 7 días	1.00
7 - 30 días	0.80
30 - 90 días	0.60
90 - 365 días	0.40
> 365 días	0.20

Evidencia (detalle)

Formula: $0.5 + 0.5 * (\text{cantidad_evidencia} / 5)$. Maximo 1.0, minimo 0.0. Ej: una decision con 3 fuentes: $0.5 + 0.5 * (3/5) = 0.8$

5. APIs MCP (FastAPI)

Servidor MCP via FastAPI para que agentes de IA y herramientas externas interactuen con la memoria de forma estandarizada.

Endpoint	Metodo	Proposito
/health	GET	Health check
/ingest	POST	Ingestar episodio
/retrieve	POST	Busqueda hibrida
/consolidate	POST	Ejecutar consolidacion
/evaluate	POST	Evaluar calidad retrieval

\$ Inicio: uvicorn ai_architect.core.mcp_server:app --reload --port 8000

Ejemplo retrieve

```
POST /retrieve
{"query": "autenticacion JWT", "top_k": 3,
 "project": "demo-auth", "collection": "semantic"}

Respuesta:
{"trace_id": "tr_4a9b...",
 "results": [{
   "id": "sem_bd51...",
   "text": "Usar JWT con refresh tokens rotativos",
   "score": 0.7497,
   "evidence": [{"type": "episode", "id": "ep_45a6..."}]
 }}
```


6. Mejoras Futuras Recomendadas

Prioridad Alta

Mejora	Impacto	Descripcion
Embeddings reales (OpenAI/ST)	Alto	Hash actual determinista, pobre semanticamente
UI/web dashboard	Alto	Visualizar memoria, confianzas y grafos de conocimiento
BM25 + embeddings hibridos	Alto	Mejor retrieval que solo vectores
Pipelines asincronos	Alto	Workers para no bloquear el servidor

Prioridad Media

Mejora	Impacto	Descripcion
Autenticacion MCP (API key + JWT)	Medio	Seguridad en produccion
Versionado semantico (supersedes)	Medio	Cuando una decision cambia
Deteccion de contradicciones con LLM	Medio	LLM-based factcheck vs heuristico
Export programado (cron/backups)	Medio	Backup periodico automatico

Prioridad Baja

Mejora	Impacto	Descripcion
Tests de integracion con ChromaDB real	Bajo	Tests actuales con hash embedding
Dockerfile + docker-compose	Bajo	Facil deploy en cualquier entorno
MCP stdio server (para OpenCode)	Bajo	Alternativa a FastAPI para agentes CLI
Agente reflector con LLM	Bajo	Reflexiones mas inteligentes que heurísticas

7. Estructura de Datos

7.1 Episodio

Cada episodio representa una experiencia o evento unico.

```
{
  "id": "ep_45a69c4296b04945...",
  "project": "demo-auth",
  "timestamp": "2026-05-14T18:24:05Z",
  "source_type": "meeting",
  "author": "ana",
  "title": "Decision de autenticacion",
  "summary": "Usar JWT con refresh tokens rotativos...",
  "decisions": [{"text": "Usar JWT...", "owner": "ana"}],
  "risks": [{"text": "Riesgo de claves", "severity": "high"}],
  "evidence": [{"type": "doc", "url_or_path": "confluence://...",
    "excerpt": "ADR-01 aprueba JWT + refresh rotativo"}],
  "tags": ["auth", "seguridad"]
}
```

7.2 Semantico (conocimiento consolidado)

```
{
  "id": "sem_bd5103e9b5db4642...",
  "type": "decision",
  "project": "demo-auth",
  "statement": "Usar JWT con refresh tokens rotativos",
  "confidence": 0.8,
  "evidence_source_ids": ["ep_45a6...", "ep_4271..."],
  "contradictions": [],
  "tags": ["auth", "seguridad"],
  "created_at": "2026-05-14T18:24:05Z"
}
```

7.3 Traza de auditoria

Toda operacion genera una traza en logs/traces/*.jsonl (JSON Lines).

```
{"trace_id": "tr_4a9b...", "ts": "2026-05-14T18:24:05Z",
  "operation": "retrieve",
  "inputs": {"query": "autenticacion", "top_k": 3},
  "outputs": {"count": 1, "results": [...]}}
```

8. Comandos de Uso

Demo completa

```
$env:PYTHONPATH="../../../brain-ai-01"
python scripts/make_demo.py
```

Servidor MCP

```
uvicorn ai_architect.core.mcp_server:app --reload --port 8000
```

Ingestar episodio

```
python scripts/ingest_cli.py --file episodio.json
```

Busqueda hibrida

```
python scripts/retrieve_cli.py --query "autenticacion JWT" --project demo-auth
```

Consolidar

```
python scripts/consolidate_cli.py --project demo-auth
```

Evaluar

```
python scripts/evaluate_cli.py --query "autenticacion JWT"
```

Tests

```
python -m pytest tests/ -v
```

Exportar backup

```
python -c "from ai_architect.pipelines.export import export_all; print(export_all())"
```

Fin del manual

Documento generado automaticamente desde el codigo fuente.